W02

Wrangling and plotting ecological data

R PROGRAMMING

Field data rarely arrive ready to analyse. Today you take three months of bird counts, fix the problems in them, reshape them into tidy format, join them to site and species information, and turn the result into summary tables and publication-quality figures.

DURATION	About 2 hours, with a short break.
YOU NEED	R and RStudio with the `tidyverse` installed, and your Week 1 project folder.
FILES	`week02_wrangling_and_plotting.R`, `week02_solutions.R`, `bird_counts_wide.csv`, `species_traits.csv`, `landcover_classes.csv`, `survey_points.csv`
DATA	The 40 points from Week 1, each visited in April, May and June 2026; counts of 10 bird species. Simulated.

Learning outcomes

By the end of this session you can:

- chain steps together with the pipe `|>`;
- find and fix inconsistent text, text dates and missing values;
- reshape data between wide and long format with `pivot_longer()` and `pivot_wider()`;
- combine tables with `left_join()`;
- summarise by group to get species richness and abundance per point and per habitat;
- build and save figures with `ggplot2`.

1 · Tidy data

A dataset is **tidy** when each variable is a column, each observation is a row, and each value is a cell. Survey sheets are usually **wide**: one column per species, because that is quick to fill in. For analysis we want them **long**: one row per point, visit and species.

Wide, as recorded in the field (first columns only):

point_id	visit	date	observer	european_robin	common_chaffinch	…
P01	1	14/04/2026	M. Puig	1	4	...
P01	2	12/05/2026	M. Puig	2	2	...

Long, ready for dplyr and ggplot2:

point_id	visit	date	observer	species	count
P01	1	2026-04-14	M. Puig	european_robin	1
P01	1	2026-04-14	M. Puig	common_chaffinch	4
...					

The pipe

`x |> f()` means "take x, and then do f to it". Pipes let you write an analysis as a sequence of steps that reads from top to bottom. Shortcut: **Ctrl + Shift + M** (**Cmd + Shift + M** on a Mac). You will also see %>% in older code; it works the same way here.

```
counts |>
  filter(visit == 1) |>          # keep the April visit
  select(point_id, woodlark) |>  # keep two columns
  arrange(desc(woodlark))        # most woodlarks first
```

2 · Cleaning: look before you analyse

The counts file contains three kinds of problem that are common in real field data. Finding them is part of the work.

Problem	How to spot it	Fix used in the script
Observer names typed seven different ways ("M. Puig", "m. puig", "M.Puig"...)	`count(observer)`	Remove spaces and dots, convert to upper case, then map each key to one spelling with `case_when()`
Dates stored as text, day/month/year	`glimpse()` shows <chr>	`dmy()` from lubridate converts them to real dates
6 counts missing (NA) for three species	`summarise(across(..., ~ sum(is.na(.))))`	Keep NA distinct from 0, and drop those rows after reshaping

```
counts <- counts |>
  mutate(
    obs_key  = str_to_upper(str_remove_all(observer, "[ .]")),  # "MPUIG"
    observer = case_when(obs_key == "MPUIG"   ~ "M. Puig",
                         obs_key == "JSERRA"  ~ "J. Serra",
                         obs_key == "AFERRER" ~ "A. Ferrer")
  ) |>
  select(-obs_key) |>
  mutate(date = dmy(date))
```

Never fix raw data by hand

It is tempting to correct typos in Excel. Do the corrections in your script instead: the raw file stays untouched, every change is documented, and if a colleague sends an updated file you rerun the script and get the same cleaning.

3 · Reshape and join

```
counts_long <- counts |>
  pivot_longer(cols = european_robin:short_toed_treecreeper,
               names_to = "species", values_to = "count") |>
  filter(!is.na(count))        # 40 x 3 x 10 = 1,200 rows, minus 6 NAs = 1,194

birds <- counts_long |>
  left_join(traits, by = "species") |>   # scientific name, family, diet...
  left_join(points, by = "point_id")     # habitat, elevation, coordinates
```

`left_join()` keeps every row of the left table and adds the matching columns from the right table. Always check the number of rows before and after a join: if it grows, the key (`species` or `point_id`) is duplicated in the right-hand table.

4 · Richness and abundance

Two classic community measures, calculated per point across the three visits:

- **Species richness**: number of the 10 surveyed species seen at least once.
- **Abundance**: total birds counted, divided by the number of visits (birds per visit).

Summarising again by habitat gives this table:

Habitat	Points	Mean richness	SD	Birds per visit
pine plantation	5	8.8	1.1	9.6
holm oak forest	9	8.4	0.7	11.7
mixed forest	6	8.0	1.1	13.2
beech forest	9	7.0	1.2	11.4
heathland	6	5.3	1.4	5.2
meadow	5	4.8	0.4	5.4

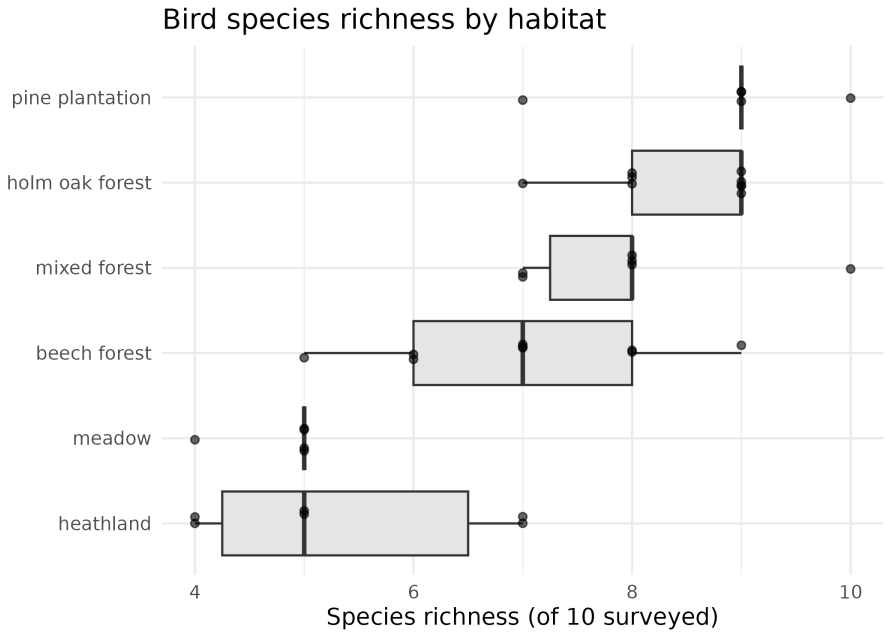
Think like an ecologist

Open habitats hold fewer species and fewer birds than forest. Before you believe that, ask: are there enough points per habitat (only 5 in meadow)? Is a bird in an open meadow as easy to detect as one in dense beech forest? Raw counts mix true abundance with detectability. We return to this with models in Weeks 3 and 4.

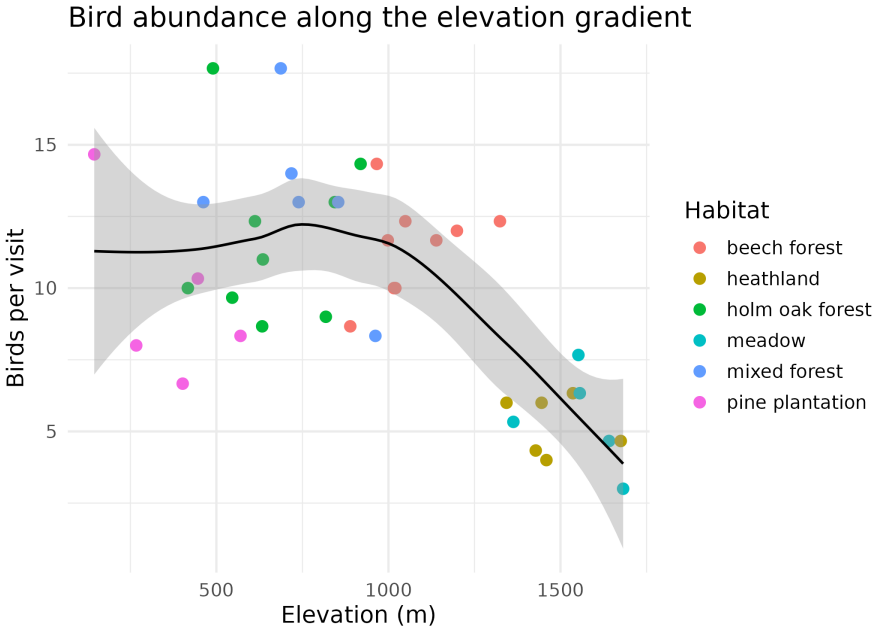
5 · ggplot2: the grammar of graphics

Every ggplot is built from the same parts: the **data**, **aesthetics** that map columns to visual properties (x, y, colour, size) and **geoms**, the marks that are drawn. You add layers with +.

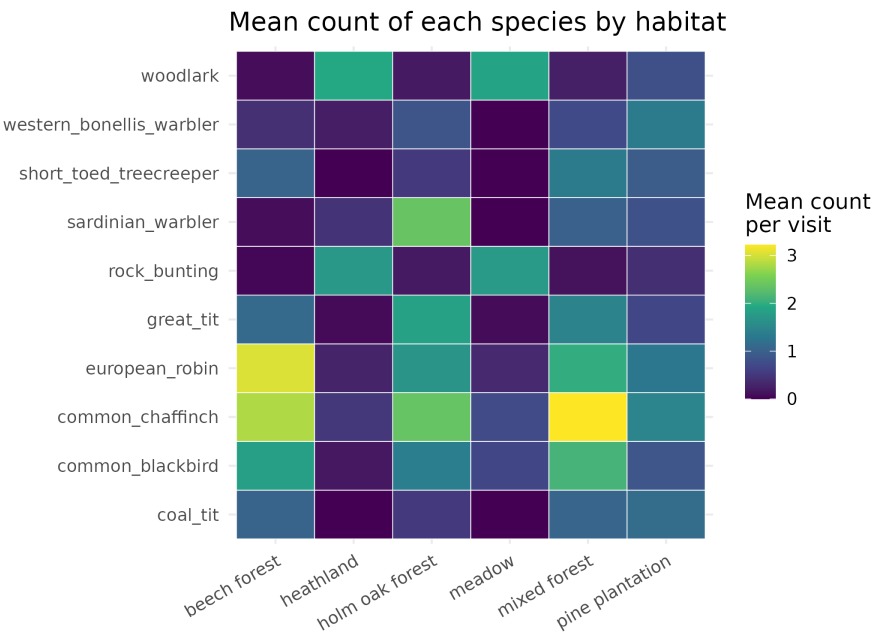
```
ggplot(point_summary, aes(x = elevation_m, y = abundance)) +   # data + aesthetics
  geom_point(aes(colour = habitat), size = 2.5) +              # layer 1: points
  geom_smooth(method = "loess", colour = "black") +            # layer 2: trend
  labs(x = "Elevation (m)", y = "Birds per visit") +           # labels with units
  theme_minimal()
```



Richness by habitat: a boxplot with the individual points on top, so the sample size is visible.



Abundance along the elevation gradient, with a loess smoother and its confidence band. Abundance drops above about 1,300 m, where forest gives way to open habitat.



Mean count of each species per visit in each habitat. Woodlark and rock bunting dominate the open habitats; chaffinch and robin dominate the forests.

Save figures from code with `ggsave()`, giving the size in inches and 300 dpi, which most journals accept. That way every figure can be recreated identically.

Key functions from today

Function	What it does
`read_csv(), write_csv()`	Read and write CSV files (readr)
`glimpse(), count()`	Inspect columns; count rows per category
`filter(), select(), arrange()`	Keep rows, keep columns, sort rows
`mutate()`	Create or change columns
`group_by() + summarise()`	One summary row per group
`pivot_longer(), pivot_wider()`	Reshape between wide and long
`left_join()`	Add columns from another table by a key
`case_when()`	Assign values by conditions
`dmy()`	Turn day/month/year text into dates (lubridate)
`ggplot() + geom_*()`	Build a plot layer by layer
`ggsave()`	Save a plot to a file

Exercises

At the end of the script. Answers are in `week02_solutions.R`.

1. How many birds did each observer record in total? Who recorded most *per visit*, and why is that the fairer comparison?
2. Find the three species with the highest total count.
3. Create a column separating forest from open habitat (`if_else()` and `%in%`) and compare mean richness.

4 Plot canopy cover against richness, coloured by habitat, with a straight-line trend (`geom_smooth(method = "lm")`).

5 Bar chart of mean count per diet group and habitat (`geom_col(position = "dodge")`).

6 **Challenge:** reshape the species-by-habitat table back to wide format and save it.

Before Week 3

- Finish the exercises and compare with the solutions.
- Optional reading: *R for Data Science*, chapters on data transformation, tidy data and joins.